

Thème : Transition, transformation, conversion

PicarX - Raspberry PI : Contrôle à Distance et IA

Problématique :

Quels défis techniques l'intégration, dans un même robot interactif, de fonctionnalités variées (suivi de visage, communication vocale bidirectionnelle, téléguidage, pilotage autonome et intelligence artificielle embarquée) soulève-t-elle ?



0. Introduction

Dans le cadre du TIPE 2024-2025, nous avons choisi de concevoir **un robot pédagogique modulaire** reposant sur un Raspberry Pi 4. Ce micro-ordinateur sous Linux¹ offre un compromis idéal : assez puissant pour exécuter de l'IA embarquée, suffisamment compact pour être intégré au châssis.

La première phase du projet a consisté à **prendre en main l'environnement Linux/Raspberry Pi** et à étudier le fonctionnement du capteur à ultrasons. Ces explorations ont révélé le potentiel – mais aussi les limites – d'un système embarqué lorsque l'on souhaite combiner vision par ordinateur, synthèse vocale, intelligence artificielle, détection de panneaux/visages, communication vocale, système sonore et contrôle en temps réel.

Quels défis techniques l'intégration, dans un même robot interactif, de fonctionnalités variées (suivi de visage, communication vocale bidirectionnelle, téléguidage, pilotage autonome et intelligence artificielle embarquée) soulève-t-elle ?

Notre objectif est de transformer le robot SunFounder Picar-X en un **véritable compagnon intelligent** (nommé ByteRacer), capable d'interagir naturellement avec l'utilisateur et son environnement. Pour répondre à la problématique, la suite du synoptique est structurée comme ceci : 1. Etude du capteur ; 2. Architecture matérielle et logicielle ; 3. Pilotage à distance ; 4. Communication vocale bidirectionnelle ; 5. Intégration de l'IA ; 6. Suivi de Visage ; 7. Reconnaissance d'un feu tricolore et de panneaux de signalisation.

I. Étude du capteur

Il s'agit d'évaluer le fonctionnement et la précision du capteur à ultrasons HC-SR04 pour la mesure de distances, en vue de son intégration dans un robot autonome. Le capteur utilise des ondes mécaniques qui se propagent dans l'air par collision de molécules. Les ultrasons, inaudibles pour l'homme (>20 kHz), sont réfléchis par un obstacle. Le capteur capte l'écho via un micro, et calcule la distance à partir du temps mesuré. Le capteur utilise deux modules piézoélectriques (émission/réception) basés sur des céramiques capables de vibrer sous tension.

Après le déclenchement de l'émission avec une impulsion de 10µs sur la broche TRIGGER, le capteur va émettre 8 impulsions à 40 kHz. Une fois l'onde réfléchi et captée par le micro la broche ECHO passe à l'état haut pendant une durée proportionnelle à la distance

$$d = \frac{c \cdot t}{2}$$

Branchement sur le Raspberry Pi :

Broche	Robot HAT ²	Raspberry Pi
TRIGGER	D2	GPIO 27
ECHO	D3	GPIO 22
5V	5V	5V
GND	GND	GND

Le capteur HC-SR04 s'est révélé **fiable et précis** dans une plage adaptée à la robotique. Malgré quelques **limites** (obstacles absorbants, angles d'incidence, portée réduite), sa **simplicité d'utilisation**, sa **faible consommation** et son **coût réduit** en font un excellent choix pour notre projet de TIPE.

d	Distance (m)
c	Vitesse du son dans l'air (m/s)
t	Durée de l'état haut sur la broche ECHO(s)

Caractéristiques :

- Plage : 2 à 400 cm
- Précision : ±0,3 cm
- Angle de détection : 15° (total 30°)
- Durée du retour 100 µs à 18 ms (36 ms si pas de retour)
- Attente de 10 ms avant nouvelle mesure
- Tension : 5V
- Courant : 15 mA
- Fréquence : 40Hz



Figure 1 : Capteur à ultrasons

II. Architecture matérielle et logicielle

A. Structure Générale de communication :

Notre système relie l'utilisateur au robot en trois étapes. D'abord, une page web fait office d'interface homme machine et affiche la vidéo, les capteurs et les commandes. Elle échange en temps réels des messages **WebSocket³** en

¹ Noyau de système d'exploitation libre de type Unix, base de nombreuses distributions.

² Carte d'extension « Hardware Attached on Top » au format Raspberry Pi (connecteur 40 broches et EEPROM) qui se superpose pour ajouter des fonctions matérielles.

³ Protocole réseau sur une seule connexion, maintenant un canal ouvert pour échanges bidirectionnels en temps réel entre navigateur et serveur.

format JSON⁴ avec un petit serveur tournant sur le Raspberry Pi. Ce serveur transmet les données au programme Python embarqué, qui pilote capteurs, moteurs et synthèse vocale, puis renvoie les informations à l'écran.

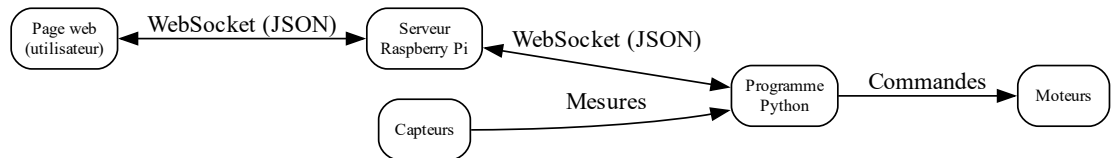


Figure 2 : Architecture simplifiée du système et circulation des données.

B. Structure interne du contrôleur Python

Le programme `main.py` lance ByteRacer puis démarre chaque service du robot dans sa propre tâche asynchrone⁵. Un module charge la configuration et tient les journaux⁶ (logs) ; d'autres lisent capteurs, caméra, vision IA et micro. Les modules d'action pilotent moteurs, sons et synthèse vocale. GPTManager interprète les requêtes et propose mouvements ou phrases, tandis que NetworkManager assure la connexion Wi-Fi ou point d'accès. Si un capteur détecte un danger, ByteRacer coupe aussitôt les moteurs, joue un son et annonce la cause, sans dépendre du réseau.

III. Pilotage à Distance

Pour piloter le robot, l'utilisateur peut associer une manette de console au site web via Bluetooth. Dès qu'elle est connectée, le navigateur lit 60 fois par seconde les valeurs des

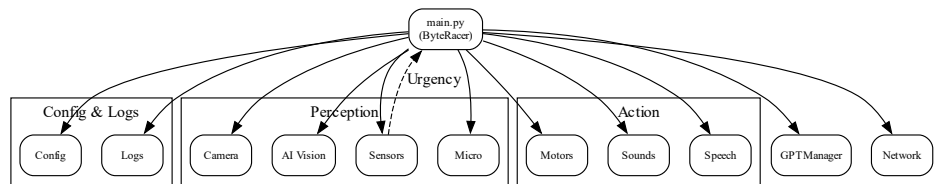


Figure 3 : Architecture simplifiée du logiciel ByteRacer.

axes et des boutons, applique les traitements nécessaires (zones mortes, inversions, limites), puis expédie ces données en JSON sur un WebSocket. La passerelle EagleControl les transmet aussitôt au Raspberry Pi. De son côté, ByteRacer lisse les vitesses, s'assure qu'aucun obstacle, trou ou niveau de batterie critique ne menace le robot, puis envoie les ordres aux moteurs pour avancer, reculer ou tourner.

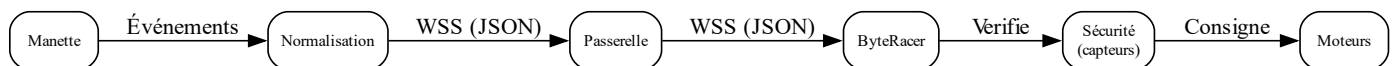


Figure 4 : Chaîne simplifiée de traitement d'une commande manette

L'aller-retour complet — manette, réseau local, contrôleur, moteurs — prend environ 70 ms et protège toujours la machine, permettant ainsi un contrôle à distance fiable et ergonomique.

IV. Communication vocale bidirectionnelle

Le système de communication emploie un seul canal WebSocket nommé `audio_stream` dans les deux sens. Le micro (PC ou robot) découpe la voix en blocs WAV⁷ de 250 ms, les encode en Base 64⁸, puis les envoie aussitôt. Cette taille maintient le son fluide sans alourdir le processeur. À l'arrivée, le récepteur décode chaque bloc et le joue immédiatement. Un horodatage⁹ dans le message élimine tout paquet vieux de plus de 750 ms pour éviter l'écho, et l'écoute du micro du robot peut être activée ou coupée à distance.

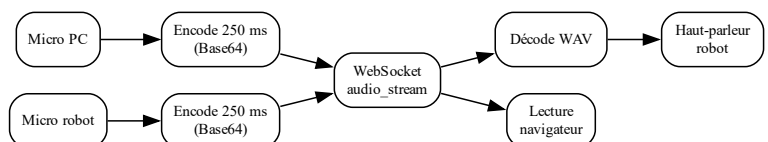


Figure 5 : Échange vocal bidirectionnel simplifié (PC ↔ robot)

V. Intégration de l'IA

La page web envoie votre instruction (texte ou voix) par WebSocket ; sur le Raspberry Pi, ByteRacer la remet à GPTManager qui l'associe, si besoin, à une photo et au contexte, interroge l'API¹⁰ OpenAI, puis interprète et exécute la réponse : déplacer le robot (séquences moteurs), exécuter instantanément un script Python embarqué, jouer un effet sonore, parler via synthèse vocale, ou lancer une action prédéfinie comme un salut — et, si la réponse ne demande rien de tout cela, simplement répondre par du texte. Chaque étape est transmise à l'interface, et dès la

⁴ JavaScript Object Notation est un format de données textuel permettant la représentation et la transmission d'information structurée.

⁵ Tâche lancée sans bloquer le programme ; son résultat arrive plus tard.

⁶ Enregistrements horodatés d'événements qu'un programme ou système produit à des fins de diagnostic, suivi et audit.

⁷ Format de fichier audio non compressé (Waveform Audio File Format, RIFF) qui stocke des échantillons PCM et leurs métadonnées.

⁸ Encodage binaire-vers-texte qui transforme des octets en un alphabet ASCII de 64 caractères, facilitant leur transmission dans des canaux textuels.

⁹ Marque qui enregistre la date et l'heure exactes associées à un événement, un message ou un fichier.

¹⁰ Interface de programmation qui expose des fonctions et formats d'échange permettant à un logiciel d'interagir avec un autre.

tâche finie, le contrôle revient au pilote. En mode conversation, le robot peut aussi écouter son propre micro, réenclencher GPTManager à chaque phrase et continuer jusqu'à ce que l'on stoppe le dialogue.

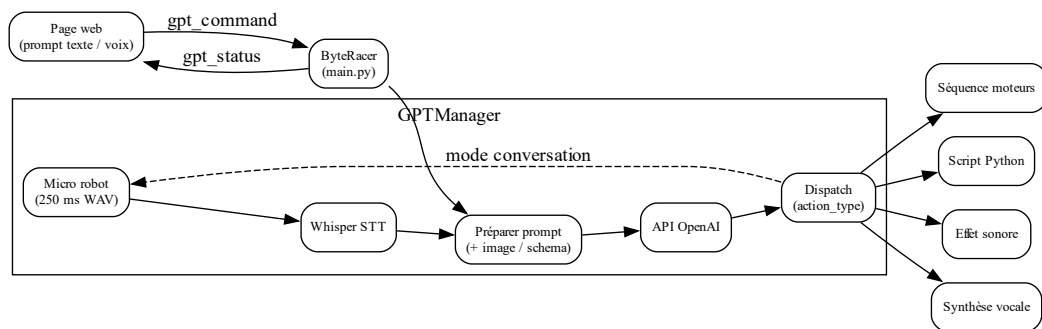


Figure 6 : Chaîne GPT simplifiée — du prompt ou de la voix jusqu'aux actions du robot

VI. Suivi de visage

Dès que l'option « Tracking » est activée, le robot ramène la caméra à l'horizontale et démarre une boucle qui, vingt fois par seconde, cherche un visage en utilisant le modèle d'IA fourni par Sunfounder. Quand il en trouve un, il oriente en douceur la caméra pour le garder au centre, estime la distance à partir de la taille du visage, puis avance ou recule afin qu'il occupe environ 10 % de l'image. Un correcteur règle en parallèle l'angle des roues pour rester face à la personne, tout en limitant vitesse et braquage pour éviter les à-coups.

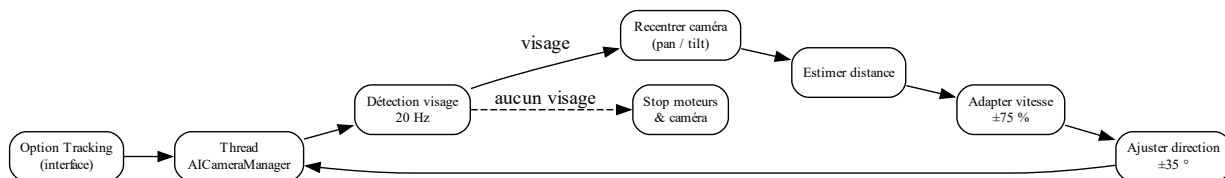


Figure 7 : Boucle simplifiée de suivi de visage.

VII. Reconnaissance d'un feu tricolore et de panneaux de signalisation

A. Création du feu tricolore :

1. Circuit électrique

Pour créer un feu tricolore qui soit autonome en énergie, nous avons utilisé un microcontrôleur esp32, une carte d'alimentation dite « step up » permettant de stabiliser la tension de la batterie li-ion 18650 à 5V et d'alimenter ainsi l'esp et des leds RGB de type WS2812. Ce type de led permet d'en connecter plusieurs en série et de pouvoir toutes les contrôler indépendamment les unes des autres avec seulement trois fils (5V, GND et data). La carte « step up » permet aussi de gérer la charge de la batterie en la connectant à un USB – C. Tous les composants ont été soudés ensemble sur une plaque d'essais.

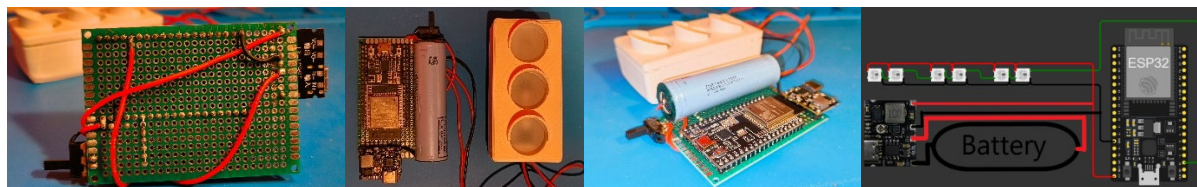


Figure 8 : Circuit électrique du feu tricolore

2. Modélisation et impression 3D

A l'aide du logiciel fusion 360, nous avons modifié un fichier 3D trouvé en ligne pour l'adapter à nos besoins.



La base du feu a été entièrement modélisée afin d'accueillir tous les composants et pour que l'USB-C (charge de la batterie), le micro USB de l'esp32 (programmation) et l'interrupteur restent accessibles. Sur la version finale de droite, nous avons réglé les problèmes rencontrés avec la version de gauche :

- Le poteau et le couvercle du boîtier ont été modifiés et collés ensembles pour une meilleure stabilité.
- Les ouvertures ont été repositionnées et redimensionnées.
- La plaque d'essais est désormais fixée par un système de mise en contrainte qui emboîte tout parfaitement.

La partie supérieure du feu diffusant les signaux lumineux a été réimprimée en PETG gris permettant de limiter grandement la diffusion de la lumière dans les zones non désirée. Au contraire, pour diffuser correctement la lumière dans les zones prévues, nous avons utilisé deux couches de papier calque superposées.

B. Création du modèle d'IA

Afin de détecter le feu de façon fiable dans de multiples environnements, nous devons concevoir un modèle d'intelligence artificielle capable de reconnaître la position et l'état du feu à partir de la caméra du robot.

1. Entrainement du modèle

Nous avons décidé de baser notre modèle sur YOLO, modèle déjà existant, spécialisé dans la reconnaissance d'objet. Pour reconnaître les états du feu nous avons pris de nombreuses photos du feu dans ses différents états, environnements et conditions lumineuses. Ensuite nous avons labellisé et encadré le feu sur chaque photo. Enfin, nous avons exporté les images labellisées pour entraîner le modèle de base. Après plusieurs heures d'entraînement, nous avons exporté un modèle assez précis, capable de détecter un feu, son état et sa position. Nous avons de même avec des panneaux de signalisation imprimés en 3D (Stop et Tourner à droite). Les graphs ci-dessous représentent l'erreur et la précision du modèle suivant différentes façons de le calculer en fonction de chaque itération de l'entraînement

2. Intégration du modèle sur le robot

Nous chargeons notre modèle sur le Raspberry Pi. Lorsque ByteRacer passe en mode circuit, on récupère le flux de la caméra dont on extrait l'image. Le modèle analyse ensuite l'image pour produire une prédiction. Si un feu ou un panneau est détecté, sa position et son état sont retournés par le modèle, il suffit alors de stopper le robot, de le faire avancer ou tourner. Finalement notre modèle est capable de détecter de façon précise et fiable les feux de circulations et les panneaux. Cependant, nous restons limités par la puissance du Raspberry Pi qui ne permet d'analyser qu'une image par seconde.



Figure 10 : Interface du logiciel de labélisation d'images.

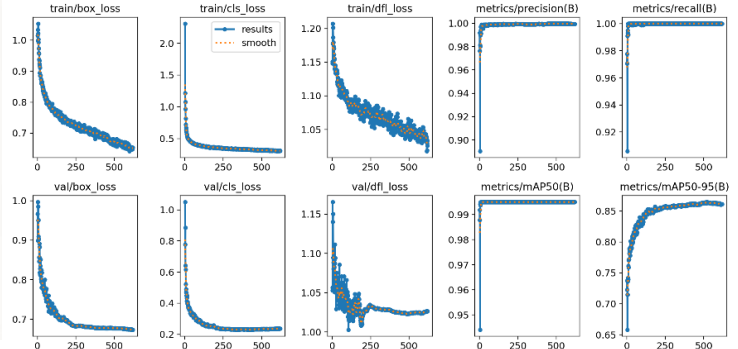


Figure 9 : Calcul de la précision du modèle en fonction de l'époque (nombre de passage complet sur le dataset)

VIII. Conclusion

Nos expérimentations nous ont montré que l'intégration de nombreuses fonctionnalités dans un système embarqué était un défi majeur. Dès le lancement du projet, nous avons peiné à trouver une direction claire ; nous avons envisagé de réaliser une « course de robot », mais nous y avons renoncé face aux défis techniques et à une complexité trop importante. Au cours de ce projet, nous avons mis en commun nos savoir-faire respectifs et renforcé nos compétences en **développement logiciel** et **matériel**, en **administration système**, ainsi qu'en **modélisation** et **impression 3D**. Cependant, le projet a été complexe à mener du fait du travail en équipe en parallèle sur un seul robot, de l'important travail de développement informatique (+ de 28 000 lignes de code !) avec des problématiques telles que la **gestion des états du robot** et l'exécution de **plusieurs tâches en parallèle**. Nous avons aussi rencontré des limites avec la **puissance de calcul** du Raspberry pour de l'IA embarquée, avec le **manque de capteurs** (nous n'avons pas d'information de distances sur les côtés ni à l'arrière), avec le **manque de chaîne de retour** sur les moteurs du robot, avec les bibliothèques¹¹ du constructeur pour la gestion du flux vidéo (nous avons dû les modifier pour les adapter à nos besoins) et avec la **latence liée au réseau** (pilotage difficile lorsque le réseau Wi-Fi est instable). Enfin, nous pourrions utiliser un Raspberry Pi 5 pour de meilleures performances d'IA et plus de capteurs permettant une meilleure visualisation 3D (par exemple LIDAR) pour permettre un mode de conduite autonome.

IX. Bibliographie :

- Documentation du robot : <https://docs.sunfounder.com/projects/picar-x-v20/fr/latest/>
- Documentation du hat : <https://docs.sunfounder.com/projects/robot-hat/en/latest/>
- Documentation de la bibliothèque de vidéo : <https://docs.sunfounder.com/projects/vilib-rpi/en/latest/>
- Documentation de l'API d'OpenAI : <https://platform.openai.com/docs/overview>
- Documentation du modèle de reconnaissance d'images : <https://docs.ultralytics.com/fr/>
- Vidéo qui nous a aidé à faire notre modèle : <https://www.youtube.com/watch?v=r0RspiLG260>

¹¹ En programmation, une bibliothèque est un ensemble regroupé de fonctions, classes ou ressources prêtes à être réutilisées pour accomplir des tâches spécifiques sans réécrire le code.